Глава 7

Коллекции

В этой главе...

- 7.1. Краткий обзор каркаса коллекций
- 7.2. Итераторы
- 7.3. Множества
- 7.4. Отображения
- 7.5. Другие коллекции
- 7.6. Представления
- Упражнения

Для программистов разработано немало структур данных, где они могут эффективно хранить значения и извлекать их. В прикладном программном интерфейсе Java API предоставляются реализации наиболее употребительных структур данных и алгоритмов обращения с ними, а также специальный каркас для их организации. В этой главе поясняется, как обращаться со списками, множествами, отображениями и прочими разновидностями коллекций.

Основные положения этой главы приведены ниже.

1. Интерфейс `Collection` предоставляет общие методы для всех коллекций, кроме отображений, которые описывает интерфейс `Map`.
2. Список представляет собой последовательную коллекцию, где каждый элемент имеет целочисленный индекс.
3. Множество оптимизировано для эффективной проверки вложенности. В языке Java предоставляются реализации множеств в виде классов `HashSet` и `TreeSet`.
4. Отображения реализованы в виде классов `HashMap` и `TreeMap`. А отображение типа `LinkedHashMap` сохраняет порядок ввода элементов.
5. Интерфейс `Collection` и класс `Collections` предоставляют немало полезных алгоритмов для операций над множествами, поиска, сортировки, перетасовки и пр.
6. Представления обеспечивают доступ к данным, хранящимся в другом месте, используя стандартные интерфейсы коллекций.

7.1. Краткий обзор каркаса коллекций

Каркас коллекций в Java предоставляет реализации общих структур данных. Чтобы упростить написание прикладного кода, зависящего от выбора структур данных, в каркасе коллекций предоставляется целый ряд общих интерфейсов (рис. 7.1). Основным среди них считается интерфейс `Collection`, методы которого перечислены в табл. 7.1.

Таблица 7.1. Методы из интерфейса **`Collection<E>`**

Метод	Описание
`boolean add(E e)`	Вводят заданную коллекцию ***e*** или элементы из заданной коллекции ***c***
`boolean addAll(Collection<? extends E> c)`	Возвращают логическое значение **`true`**, если коллекция изменилась
`boolean remove(Object o)`	Удаляют заданную коллекцию ***o***, элементы из заданной коллекции ***c***,
`boolean removeAll(Collection<?> c)`	элементы, отсутствующие в заданной коллекции ***c***, совпадающие или все элементы
`boolean retainAll(Collection<?> c)`	Первые четыре метода возвращают логическое значение **`true`**, если коллекция изменилась
`boolean removeIf(Predicate<? super E> filter)`	

Окончание табл. 7.1

Метод	Описание
`void clear()`	
`int size()`	Возвращает количество элементов в данной коллекции
`boolean isEmpty()`	Возвращают логическое значение `true`, если коллекция пуста,
`boolean contains(Object o)`	содержит заданную коллекцию *o*
`boolean containsAll(Collection<?> c)`	или все элементы заданной коллекции *c*
`Iterator<E> iterator()`	Возвращают итератор, поток данных, а возможно, и параллельный поток данных или итератор-разделитель для перебора элементов данной коллекции
`Stream<E> stream()`	Подробнее об итераторах см. в разделе 7.2, а о потоках данных — в главе 8
`Stream<E> parallelStream()`	Итераторы-разделители представляют интерес только для разработчиков потоков данных
`Spliterator<E> spliterator()`	
`Object[] toArray()`	Возвращает массив, состоящий из элементов данной коллекции
`T[] toArray(T[] a)`	Возвращает массив a, состоящий из элементов данной коллекции, если у него достаточная длина

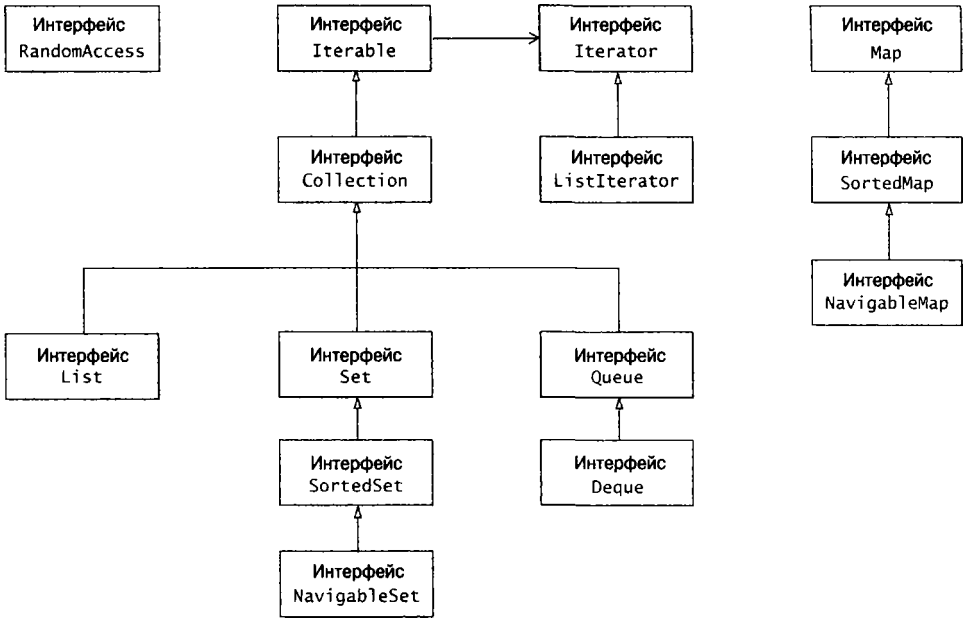


Рис. 7.1. Интерфейсы из каркаса коллекций в Java

Интерфейс List представляет последовательную коллекцию, состоящую из элементов на позициях 0, 1, 2 и т.д. В табл. 7.2 перечислены методы из этого интерфейса.

Таблица 7.2. Методы из интерфейса List

Метод	Описание
`boolean add(int index, E e)`	Вводят заданную коллекцию *e* или элементы из заданной коллекции *c*, до указанной позиции *index* или до конца коллекции
`boolean add(int index, Collection<? extends E> c)`	Возвращают логическое значение `true`, если список изменился
`boolean add(E e)`	
`boolean add(Collection <? extends E> c)`	
`E get(int index)`	Получают, устанавливают или удаляют элемент из списка по указанному индексу
`E set(int index, E element)`	Два последних метода возвращают элемент, находившийся в списке по указанному индексу перед их вызовом
`E remove(int index)`	
`int indexOf(Object o)`	Возвращают индекс первого или последнего элемента в списке,
`int lastIndexOf(Object o)`	равный 0 или `-1`, если отсутствует совпадение
`ListIterator<E> listIterator()`	Возвращают список итераторов для всех элементов или части элементов,
`ListIterator<E> listIterator(int index)`	начиная с указанной позиции *index*
`void replaceAll(Unary Operator<E> operator)`	Заменяет каждый элемент результатом выполнения над ним указанной операции
`void sort(Comparator <? super E> c)`	Сортирует этот список, используя заданное упорядочение *c*
`List<E> subList(int fromIndex, int toIndex)`	Возвращает представление (см. раздел 7.6) подсписка, начиная с заданной позиции *fromIndex* и до позиции, предшествующей заданной позиции *toIndex*

Интерфейс List реализуется классом ArrayList, представленным ранее в данной книге, а также классом LinkedList. Если вы когда-нибудь изучали структуры данных, то, вероятно, помните, что связный список — это последовательность связанных вместе узлов, каждый из которых содержит элемент данного списка. Чтобы быстро ввести элемент посредине связного списка, достаточно нарастить узел. Но для того чтобы добраться до средины связного списка, придется отследить все связи с самого его начала, что замедляет дело. Связные списки, конечно, находят свое применение, но большинство прикладных программистов предпочитают списочные массивы, если им требуется последовательная коллекция. Тем не менее интерфейс List полезен. Например, метод Collections.nCopies(n, o) возвращает объект типа List с n копиями объекта o. Этот объект вводит в заблуждение тем, что он на самом деле не хранит n копий, но когда запрашивается любая из них, возвращает объект o.



ВНИМАНИЕ. Интерфейс **List** предоставляет методы для доступа к *n*-му элементу списка, даже если такой доступ может оказаться неэффективным. Чтобы указать на наличие такого доступа, класс коллекции должен реализовать маркерный интерфейс **RandomAccess** без методов. Например, класс **ArrayList** реализует интерфейсы **List** и **RandomAccess**, тогда как класс **LinkedList** — только интерфейс **List**.

В множество, представленное интерфейсом `Set`, элементы не вводятся на конкретной позиции, а дублирующиеся элементы не допускаются. Множество, представленное интерфейсом `SortedSet`, допускает итерацию своих элементов по порядку сортировки, а в интерфейсе `NavigableSet` имеются методы для поиска соседних элементов в множестве. Подробнее о множествах речь пойдет в разделе 7.3.

Очередь, представленная интерфейсом `Queue`, сохраняет порядок ввода в нее элементов, но их можно вводить только в хвост очереди, а удалять — из ее головы (подобно живой очереди). Очередь, представленная интерфейсом `Deque`, имеет двусторонний доступ, т.е. элементы вводятся и удаляются из нее с обоих ее концов.

Интерфейсы всех коллекций являются обобщенными, причем параметр типа обозначает тип элементов коллекции (например, `Collection<E>`, `List<E>` и т.д.). В интерфейсе `Map<K, V>` параметр типа `K` обозначает тип ключа, а параметр типа `V` — тип значения.

Рекомендуется как можно больше пользоваться интерфейсами в прикладном коде. Например, после построения списочного массива типа `ArrayList` ссылку на него нужно сохранить в переменной типа `List` следующим образом:

```
List<String> words = new ArrayList<>();
```

Всякий раз, кода реализуется метод, обрабатывающий коллекцию, в качестве параметра типа следует выбирать наименее ограничительный интерфейс. Для этой цели лучше всего подходит интерфейс `Collection`, `List` или `Map`.

Одно из преимуществ каркаса коллекций заключается в том, что он избавляет от необходимости изобретать колесо, предоставляя общие алгоритмы обращения с коллекциями. Некоторые из основных алгоритмов реализованы в методах (например, `addAll()` и `removeIf()`) из интерфейса `Collection`. Служебный класс `Collections` предоставляет немало дополнительных алгоритмов для обращения с разными видами коллекций. Они позволяют сортировать, тасовать, циклически сдвигать и обращать списки, находить максимальное и минимальное значения или позицию произвольного элемента в коллекции и формировать коллекции без элементов, с одним элементом или `n` копиями одного и того же элемента. Наиболее употребительные методы из класса `Collections` сведены в табл. 7.3.

Таблица. 7.3. Полезные методы из класса **Collections**

Метод (все методы статические)	Описание
boolean disjoint(Collection<?> *c1*, Collection<?> *c2*)	Возвращает логическое значение **true**, если в сравниваемых коллекциях отсутствуют общие элементы
boolean addAll(Collection<? super T> *c*, T... *elements*)	Вводит все элементы в заданную коллекцию *c*

Продолжение табл. 7.3

Метод (все методы статические)	Описание
`void copy(List<? super T> dest, List<? extends T> src)`	Копирует все элементы из исходной коллекции *src* в целевую коллекцию *dest*, которая должна быть не меньше исходной, по тем же самым индексам
`boolean replaceAll(List<T> list, T oldVal, T newVal)`	Заменяет все прежние элементы *oldVal* новыми элементами *newVal*, каждый из которых может быть пустым (`null`). Возвращает логическое значение `true`, если найдено хотя бы одно совпадение
`void fill(List<? super T> list, T obj)`	Устанавливает указанный объект *obj* во всех элементах списка
`List<T> nCopies(int n, T o)`	Получает неизменяемый список, состоящий из *n* копий указанного объекта *o*
`int frequency(Collection<?> c, Object o)`	Возвращает количество элементов из заданной коллекции, равных указанному объекту *o*
`int indexOfSubList(List<?> source, List<?> target)`	Возвращают начало первого или последнего вхождения целевого списка в исходный список,
`int lastIndexOfSubList(List<?> source, List<?> target)`	а в их отсутствие — значение `-1`
`int binarySearch(List<? extends Comparable<? super T>> list, T key)`	Возвращают позицию ключа, при условии, что список отсортирован в естественном порядке следования элементов или заданном порядке *c*
`int binarySearch(List<? extends T> list, T key, Comparator<? super T> c)`	Если ключ отсутствует, возвращается разность `-i - 1`, где `i` — позиция, на которой должен быть введен ключ
`sort(List<T> list)`	Сортируют заданный список в естественном порядке следования элементов
`sort(List<T> list, Comparator<? super T> c)`	или заданном порядке *c*
`void swap(List<?> list, int i, int j)`	Переставляет элементы заданного списка на указанных позициях
`void rotate(List<?> list, int distance)`	Выполняет циклический сдвиг списка, перемещая элемент с позиции по индексу *i* на позицию по индексу `(i + distance) % list.size()`
`void reverse(List<?> list)`	Обращают или
`void shuffle(List<?> list)`	произвольно перетасовывают
`void shuffle(List<?> list, Random rnd)`	заданный список
`Set<T> singleton(T o)`	Получают одноэлементное множество,
`List<T> singletonList(T o)`	список или отображение
`Map<K, V> singletonMap(K key, V value)`	
`empty(List|Set|SortedSet|NavigableSet|Map|SortedMap|NavigableMap)()`	Получает пустое представление (см. далее раздел 7.6)
`synchronized(Collection|List|Set|SortedSet|NavigableSet|Map|SortedMap|NavigableMap)()`	Получает синхронизированное представление (см. далее раздел 7.6)

Окончание табл. 7.3

Метод (все методы статические)	Описание
`unmodifiable(Collection|List|Set|SortedSet|NavigableSet|Map|SortedMap|NavigableMap)()`	Получает неизменяемое представление (см. далее раздел 7.6)
`checked(Collection|List|Set|SortedSet|NavigableSet|Map|SortedMap|NavigableMap|Queue)()`	Получает проверяемое представление (см. далее раздел 7.6)

7.2. Итераторы

Каждая коллекция предоставляет свой способ перебора ее элементов в определенном порядке. Для этой цели в интерфейсе `Iterable<T>`, служащем супертипом для интерфейса `Collection`, определяется следующий метод:

```
Iterator<T> iterator()
```

Этот метод получает итератор, которым можно пользоваться для обхода всех элементов коллекции, как показано ниже.

```
Collection<String> coll = ...;
Iterator<String> iter = coll.iterator();
  while (iter.hasNext()) {
    String element = iter.next();
    Обработать элемент коллекции element
}
```

В данном случае достаточно воспользоваться расширенным циклом `for` следующим образом:

```
for (String element : coll) {
  Обработать элемент коллекции element
}
```



НА ЗАМЕТКУ. Для *любого* объекта **c** класса, реализующего интерфейс **`Iterable<E>`**, расширенный цикл **`for`** преобразуется в приведенную выше форму.

У интерфейса `Iterator` имеется также метод `remove()`, удаляющий перебранный ранее элемент. Так, в следующем цикле из коллекции удаляются все элементы, удовлетворяющие заданному условию:

```
while (iter.hasNext()) {
  String element = iter.next();
  if (element удовлетворяет заданному условию)
    iter.remove();
}
```

Но для этой цели проще воспользоваться методом `removeIf()` следующим образом:

```
coll.removeIf(e -> e удовлетворяет заданному условию);
```



ВНИМАНИЕ. Метод `remove()` удаляет из коллекции последний элемент, возвращаемый итератором, а не тот элемент, на который указывает итератор. Поэтому вызов метода `remove()` следует непременно перемежать с вызовом метода `next()` или `previous()`, а не вызывать его два раза подряд.

Интерфейс `ListIterator` является подчиненным интерфейсу `Iterator` и содержит методы для ввода элемента перед итератором, установки другого значения в перебираемом элементе и перемещения в коллекции назад. Этот интерфейс наиболее удобен для обращения со связными списками.

```
List<String> friends = new LinkedList<>();
ListIterator<String> iter = friends.listIterator();
iter.add("Fred"); // Fred |
iter.add("Wilma"); // Fred Wilma |
iter.previous(); // Fred | Wilma
iter.set("Barney"); // Fred | Barney
```



ВНИМАНИЕ. Если для обхода структуры данных имеется несколько итераторов и один из них изменяет ее, то другой может стать недействительным. Если же и дальше пользоваться недействительным итератором, он может сгенерировать исключение типа `ConcurrentModificationException`.

7.3. Множества

Множество, по существу, проверяет, является ли указанное значение его элементом, но оно дает и кое-что взамен, не запоминая порядок, в котором были введены элементы. Множества удобны в тех случаях, когда порядок расположения элементов не имеет значения. Так, если требуется множество неприличных слов, запрещенных в качестве имени пользователя, то порядок их расположения в множестве особого значения не имеет. Нужно лишь выяснить, находится предлагаемое имя пользователя в этом множестве.

Интерфейс `Set`, представляющий множество, реализуется классами `HashSet` и `TreeSet`. Но внутренняя реализация данного интерфейса в обоих классах заметно отличается. Если вам приходилось прежде изучать структуры данных, то вы, вероятно, знаете, как реализуются хеш-таблицы и двоичные деревья. Впрочем, пользоваться классами `HashSet` и `TreeSet` можно, даже не зная их внутренней реализации.

Как правило, хеш-множество оказывается более эффективным, при условии, что имеется подходящая *хеш-функция* для его элементов. Такие библиотечные классы, как `String` или `Path`, имеют подходящие хеш-функции. О том, как писать подходящие хеш-функции для своих классов, см. в главе 4.

Например, упомянутое выше множество неприличных слов может быть реализовано следующим образом:

```
Set<String> badWords = new HashSet<>();
badWords.add("sex");
badWords.add("drugs");
badWords.add("c++");
```

```
if (badWords.contains(username.toLowerCase()))
  System.out.println("Please choose a different user name");
```

Если множество требуется обойти в отсортированном порядке, то для этой цели следует воспользоваться классом `TreeSet`. Одной из причин для этого может служить предоставление пользователям отсортированного списка вариантов выбора.

Тип элемента множества должен относиться к классу, реализующему интерфейс `Comparable`. В противном случае в конструкторе класса придется указать интерфейс `Comparator`. Ниже приведен пример построения древовидного множества типа `TreeSet`.

```
TreeSet<String> countries = new TreeSet<>(); // Обходит названия стран,
       // введенные в древовидное множество, в отсортированном порядке
countries = new TreeSet<>((u, v) ->
    u.equals(v) ? 0
    : u.equals("USA") ? -1
    : v.equals("USA") ? 1
    : u.compareTo(v));
    // Название "USA" всегда следует первым
```

Класс `TreeSet` реализует интерфейсы `SortedSet` и `NavigableSet`, методы которых перечислены в табл. 7.4 и 7.5 соответственно.

Таблица 7.4. Методы из интерфейса **`SortedSet<E>`**

Метод	Описание
`E first()`	Возвращают первый и последний элементы в множестве
`E last()`	
`SortedSet<E> headSet(E toElement)`	Возвращают представление элементов множества,
`SortedSet<E> subSet(E fromElement, E toElement)`	начиная с заданной позиции ***`fromElement`*** и до позиции,
`SortedSet<E> tailSet(E fromElement)`	предшествующей заданной позиции ***`toElement`***

Таблица 7.5. Методы из интерфейса **`NavigableSet<E>`**

Метод	Описание
`E higher(E e)`	Возвращают из множества ближайший элемент больше,
`E ceiling(E e)`	больше или равный,
`E floor(E e)`	меньше или равный,
`E lower(E e)`	меньше заданного элемента ***`e`***
`E pollFirst()`	Удаляет и возвращает из множества первый или последний элемент,
`E pollLast()`	а если множество оказывается пустым — пустое значение **`null`**

Окончание табл. 7.5

Метод	Описание
NavigableSet<E> headSet(E *toElement*, boolean *inclusive*)	Возвращают представление элементов множества,
NavigableSet<E> subSet(E *fromElement*, boolean *fromInclusive*, E *toElement*, boolean *toExclusive*)	начиная с позиции ***fromElement*** и до позиции ***toElement***
NavigableSet<E> tailSet(E *fromElement*, boolean *inclusive*)	(включительно или исключительно)

7.4. Отображения

В отображениях хранятся связи ключей со значениями. Чтобы ввести новую связь в отображение или изменить в нем значение по существующему ключу, достаточно вызвать метод `put()`:

```
Map<String, Integer> counts = new HashMap<>();
counts.put("Alice", 1); // Вводит пару "ключ-значение" в отображение
counts.put("Alice", 2); // Обновляет значение по заданному ключу
```

В данном примере применяется хеш-отображение, которое представлено классом `HashMap` и, как хеш-множество, больше подходит в том случае, если ключи не требуется обходить в отсортированном порядке. В противном случае следует выбрать древовидное отображение, представленное классом `TreeMap`. Ниже показано, как получить значение, связанное с ключом в отображении:

```
int count = counts.get("Alice");
```

Если ключ отсутствует, метод `get()` возвращает пустое значение `null`. В данном примере это вызовет исключение типа `NullPointerException`, если искомое значение не распаковано. Поэтому лучше воспользоваться приведенным ниже методом, возвращающим подсчет `0`, если ключ отсутствует.

```
int count = counts.getOrDefault("Alice", 0);
```

Если счетчик обновляется в отображении, то сначала нужно проверить его наличие, и если он присутствует, то добавить `1` к существующему значению. Метод `merge()` упрощает эту типичную операцию. Так, в результате вызова

```
counts.merge(word, 1, Integer::sum);
```

ключ `word` связывается со значение `1`, если он прежде отсутствовал, а иначе — к предыдущему значению добавляется `1` с помощью функции `Integer::sum()`.

Методы, реализующие операции с отображением, сведены в табл. 7.6.

Таблица 7.6. Методы из интерфейса **Map<K, V>**

Метод	Описание
V get(Object *key*)	Если заданный ключ ***key*** связан с непустым значением **v**, то возвращает значение **v**,

Продолжение табл. 7.6

Метод	Описание
`V getOrDefault(Object key, V defaultValue)`	а иначе — пустое значение `null` или указанное значение по умолчанию *defaultValue*
`V put(K key, V value)`	Если заданный ключ *key* связан с непустым значением *v*, то связывает ключ *key* с указанным значением *value* и возвращает значение *v*, а иначе вводит запись и возвращает пустое значение `null`
`V putIfAbsent(K key, V value)`	Если заданный ключ *key* связан с непустым значением *v*, то игнорирует указанное значение *value* и возвращает значение *v*, а иначе вводит запись и возвращает пустое значение `null`
`V merge(K key, V value, BiFunction<? super V,? super V,? extends V> remappingFunction)`	Если заданный ключ *key* связан с непустым значением *v*, то применяет указанную функцию к значению *v* и указанному значению *value*, а заданный ключ *key* связывается с полученным результатом или удаляется, если результат пустой (`null`). В противном случае связывает заданный ключ *key* с указанным значением *value*. Возвращает результат вызова `get(key)`
`V compute(K key, BiFunction< ? super K,? super V,? extends V> remappingFunction)`	Применяет указанную функцию к заданному ключу *key* и результату вызова `get(key)`. Заданный ключ *key* связывается с полученным результатом или удаляется, если результат пустой (`null`). Возвращает результат вызова `get(key)`
`V computeIfPresent(K key, BiFunction< ? super K,? super V,? extends V> remappingFunction)`	Если заданный ключ *key* связан с непустым значением *v*, то применяет указанную функцию к заданному ключу *key* и значению *v*. Заданный ключ *key* связывается с полученным результатом или удаляется, если результат пустой (`null`). Возвращает результат вызова `get(key)`
`V computeIfAbsent(K key, Function<? super K,? extends V> remappingFunction)`	Применяет указанную функцию к заданному ключу *key*, если только он не связан с непустым значением *v*. Заданный ключ *key* связывается с полученным результатом или удаляется, если результат пустой (`null`). Возвращает результат вызова `get(key)`
`void putAll(Map<? extends K, ? extends V> m)`	Вводит все записи из указанного отображения *m*
`V remove(Object key)`	Удаляет заданный ключ и связанное с ним значение или заменяет прежнее значение
`V replace(K key, V newValue)`	Возвращают прежнее значение, а если оно отсутствует — пустое значение `null`
`boolean remove(Object key, Object value)`	Удаляют запись из отображения или заменяют прежнее значение и возвращают логическое значение `true`, при условии, что заданный ключ *key* связан с указанным значением *value*. В противном случае ничего не делают и возвращают логическое значение `false`
`boolean replace(K key, V value, V newValue)`	Эти методы представляют интерес в основном для параллельного доступа к отображению
`int size()`	Возвращает количество записей в отображении

Окончание табл. 7.6

Метод	Описание
`boolean isEmpty()`	Проверяет, является ли отображение пустым
`void clear()`	Удаляет все записи из отображения
`void forEach(BiConsumer<? super K, ? super V> action)`	Выполняет заданное действие `action` над всеми записями в отображении
`void replaceAll(BiFunction<? super K, ? super V, ? extends V> function)`	Вызывает указанную функцию для всех записей в отображении. Связывает ключи с непустыми результатами и удаляет ключи с пустыми (`null`) результатами
`boolean containsKey(Object key)`	Проверяют, содержит ли отображение заданный ключ или указанное значение
`boolean containsValue(Object value)`	
`Set<K> keySet()`	Возвращают представления ключей,
`Collection<V> values()`	значения и записи из отображения
`Set<Map.Entry<K, V>> entrySet()`	

Вызвав следующие методы, можно получить *представления* ключей и записей в отображении:

```
Set<K> keySet()
Set<Map.Entry<K, V>> entrySet()
Collection<K> values()
```

Возвращаемые в итоге коллекции не являются копиями данных из отображения, но связаны с отображением. Если удалить ключ или запись из представления, эта запись удаляется также из лежащего в основе отображения.

Чтобы перебрать все ключи и значения в отображении, достаточно перебрать множество, возвращаемое методом `entrySet()`, как показано ниже.

```
for (Map.Entry<String, Integer> entry : counts.entrySet()) {
  String k = entry.getKey();
  Integer v = entry.getValue();
  Обработать пару "ключ-значение"  k, v
}
```

С другой стороны, достаточно вызвать метод `forEach()` следующим образом:

```
counts.forEach((k, v) -> {
  Обработать пару "ключ-значение" k, v
});
```



ВНИМАНИЕ. В некоторых реализациях отображений (например, в классе **ConcurrentHashMap**) пустые значения **null** ключей или значений не допускаются. А в тех реализациях отображений, где они разрешаются (например, в классе **HashMap**), пользоваться пустыми значениями **null** следует очень аккуратно. Ведь целый ряд методов интерпретирует пустое значение **null** как признак отсутствия записи в отображении или потребность удалить ее.



СОВЕТ. Иногда ключи в отображении требуется представить не в порядке сортировки, а в другом порядке. Например, в каркасе JavaServer Faces с помощью отображения указываются метки и значения в окне выбора. Пользователи будут неприятно удивлены, если варианты выбора окажутся отсортированными по алфавиту (например, воскресенье, вторник, понедельник, пятница, среда, суббота, четверг) или хеш-коду. В таком случае следует воспользоваться классом `LinkedHashMap`, запоминающим порядок, в котором записи вводились в отображение, и перебирающим их в этом порядке.

7.5. Другие коллекции

В последующих разделах вкратце обсуждаются классы ряда других коллекций. Они могут оказаться полезными в практике программирования на Java.

7.5.1. Свойства

Класс `Properties` реализует отображение, которое легко сохраняется и загружается в простом текстовом формате. Такие отображения обычно применяются для хранения вариантов конфигурации или свойств прикладных программ, как демонстрируется в следующем примере кода:

```
Properties settings = new Properties();
settings.put("width", "200");
settings.put("title", "Hello, World!");
try (OutputStream out = Files.newOutputStream(path)) {
  settings.store(out, "Program Properties");
}
```

В итоге получается следующий файл конфигурации программы:

```
#Свойства программы
#Mon Nov 03 20:52:33 CET 2014
width=200
title=Hello, World\!
```



ВНИМАНИЕ. Файлы свойств содержат текст в коде ASCII, а не в кодировке UTF-8. Комментарии начинаются со знака `#` или `!`. Символы с кодом меньше `'\u0021'` и больше `'\u007e'` обозначаются в Юникоде как экранируемые по следующему образцу: `\`*`unnnn`*. Знак новой строки в ключе или значении обозначается как `\n`. А знаки `\`, `#`, `!` экранируются как `\\`, `\#`, `\!`.

Чтобы загрузить свойства из файла конфигурации, достаточно сделать следующий вызов:

```
try (InputStream in = Files.newInputStream(path)) {
  settings.load(in);
}
```

А для получения значения по заданном ключу следует вызвать метод `getProperty()`. Значение, используемое по умолчанию в отсутствие ключа, можно указать следующим образом:

```
String title = settings.getProperty("title", "New Document");
```



НА ЗАМЕТКУ. Исторически сложилось так, что класс **Properties** реализует интерфейс **Map<Object, Object>**, несмотря на то, что значения в отображении, которое он представляет, всегда являются символьными строками. Следовательно, для их получения *не* следует пользоваться методом **get()**, поскольку он возвращает значение в виде объекта типа **Object**.

Метод System.getProperties() возвращает объект типа Properties с системными свойствами. Наиболее употребительные системные свойства перечислены в табл. 7.7.

Таблица 7.7. Наиболее употребительные системные свойства

Ключ свойства	Описание
user.dir	Текущий рабочий каталог для данной виртуальной машины
user.home	Начальный каталог пользователя
user.name	Наименование учетной записи пользователя
java.version	Исполняемая версия Java для данной виртуальной машины
java.home	Начальный каталог установки Java
java.class.path	Путь к файлу класса, с помощью которого была запущена данная виртуальная машина
java.io.tmpdir	Каталог для хранения временных файлов (например, **/tmp**)
os.name	Наименование операционной системы (например, **Linux**)
os.arch	Архитектура операционной системы (например, **amd64**)
os.version	Версия операционной системы (например, **3.13.0-34-generic**)
file.separator	Разделитель файлов (знак / в Unix, знак \ в Windows)
path.separator	Разделитель путей к файлам (знак : в Unix, знак ; в Windows)
line.separator	Разделитель новых строк (знаки **\n** в Unix, знаки **\r\n** в Windows)

7.5.2. Множества битов

Класс BitSet представляет множество для хранения последовательности битов. Множество битов размещает их в массиве значений типа long, и поэтому пользоваться таким множеством эффективнее, чем массивом логических значений типа boolean. Множества битов удобны для хранения последовательностей двоичных разрядов признаков (или так называемых флагов) или для представления множеств неотрицательных целочисленных значений, где i-й бит, равный 1, обозначает, что значение i содержится в множестве.

В классе BitSet предоставляются удобные методы для получения и установки отдельных битов в множестве. Это намного проще, чем манипулировать отдельными битами для их хранения в переменных типа int или long. Имеются также методы, манипулирующие всеми битами в таких операциях с множествами, как объединение и пересечение. Полный перечень методов из класса BitSet приведен в табл. 7.8. Следует, однако, иметь в виду, что класс BitSet не относится к категории классов коллекций, поскольку он не реализует интерфейс Collection<Integer>.

Таблица 7.8. Методы из класса `BitSet`

Метод	Описание
`BitSet()`	Строят множество битов, которое может первоначально содержать 64 бита
`BitSet(int nbits)`	или заданное количество *nbits* битов
`void set(int bitIndex)`	Устанавливают отдельный бит в 1, или указанное значение *value* по заданному индексу,
`void set(int fromIndex, int toIndex)`	или в пределах от *fromIndex* (включительно)
`void set(int bitIndex, boolean value)`	до *toIndex* (исключительно)
`void set(int fromIndex, int toIndex, boolean value)`	Сбрасывают в 0 отдельный бит по заданному индексу
`void clear(int bitIndex)`	в пределах от *fromIndex* (включительно)
`void clear(int fromIndex, int toIndex)`	до *toIndex* (исключительно)
`void clear()`	или же все биты
`void flip(int bitIndex)`	Сменяют на обратное состояние отдельного бита по заданному индексу или
`void flip(int fromIndex, int toIndex)`	в пределах от *fromIndex* (включительно) до *toIndex* (исключительно)
`boolean get(int bitIndex)`	Получают отдельный бит по заданному индексу или
`BitSet get(int fromIndex, int toIndex)`	в пределах от *fromIndex* (включительно) до *toIndex* (исключительно)
`int nextSetBit(int fromIndex)`	Возвращают индекс следующего или предыдущего бита,
`int previousSetBit(int fromIndex)`	устанавливаемого в 1 или сбрасываемого в 0,
`int nextClearBit(int fromIndex)`	или же значение -1, если такой бит отсутствует
`int previousClearBit(int fromIndex)`	
`void and(Bitset set)`	Образует пересечение, разность,
`void andNot(Bitset set)`	объединение, симметричную разность
`void or(Bitset set)`	с заданным множеством *set*
`void xor(Bitset set)`	
`int cardinality()`	Возвращает количеств битов, установленных в данном множестве в 1. *Предупреждение*: метод `size()` возвращает текущую длину битового вектора, а не размер множества
`byte[] toByteArray[]`	Размещают биты из данного множества
`long[] toByteArray[]`	в массиве
`IntStream stream()`	Возвращают поток данных или символьную строку целочисленных значений в данном множестве
`String toString()`	(т.е. индексы битов, установленных в 1)

Окончание табл. 7.8

Метод	Описание
`static BitSet valueOf(byte[] bytes)`	Получают множество битов, содержащее предоставляемые биты
`static BitSet valueOf(long[] longs)`	
`static BitSet valueOf(ByteBuffer bb)`	
`static BitSet valueOf(LongBuffer lb)`	
`boolean isEmpty()`	Проверяют, является ли данное множество пустым
`boolean intersects(BitSet set)`	или содержит элемент, общий с заданным множеством *set*

7.5.3. Перечислимые множества и отображения

Для накопления множеств значений перечислимого типа вместо класса `BitSet` используется класс `EnumSet`. У класса `EnumSet` отсутствуют открытые конструкторы. Для построения перечислимого множества служит статический фабричный метод, как показано ниже. Для обращения с множеством типа `EnumSet` можно также пользоваться методами из интерфейса `Set`.

```
enum Weekday { MONDAY, TUESDAY, WEDNESDAY, THURSDAY,
               FRIDAY, SATURDAY, SUNDAY };
Set<Weekday> always = EnumSet.allOf(Weekday.class);
Set<Weekday> never = EnumSet.noneOf(Weekday.class);
Set<Weekday> workday = EnumSet.range(Weekday.MONDAY,
                                     Weekday.FRIDAY);
Set<Weekday> mwf = EnumSet.of(Weekday.MONDAY, Weekday.WEDNESDAY,
                              Weekday.FRIDAY);
```

Класс `EnumMap` представляет отображение с ключами, относящимися к перечислимому типу. Такое отображение реализуется как массив значений. Тип ключей указывается в конструкторе данного класса следующим образом:

```
EnumMap<Weekday, String> personInCharge = new EnumMap<>(Weekday.class);
personInCharge.put(Weekday.MONDAY, "Fred");
```

7.5.4. Стеки и разнотипные очереди

Стек — это структура данных для ввода и удаления элементов с одного конца, называемого "вершиной" стека. Обычная очередь позволяет вводить в нее элементы с одного конца, называемого "хвостом", и удалять их с другого конца, называемого "головой" очереди. Очередь с двухсторонним доступом (или двухсторонняя очередь) допускает ввод и удаление элементов с обоих ее концов. Ввод элементов посредине всех этих структур данных не предусмотрен.

Методы для обращения с упомянутыми выше структурами данных определяются в интерфейсах `Queue` и `Deque`. Но для стека в каркасе коллекций предусмотрен не интерфейс, а только класс `Stack`, внедренный на самой ранней стадии существования Java, и поэтому пользоваться им не рекомендуется. Если требуется организовать стек,

обычную или двухстороннюю очередь, а потоковая безопасность особого значения не имеет, то для этой цели можно воспользоваться классом ArrayDeque.

Для обращения со стеком служат методы push() и pop(), как показано ниже.

```
ArrayDeque<String> stack = new ArrayDeque<>();
stack.push("Peter");
stack.push("Paul");
stack.push("Mary");
while (!stack.isEmpty())
  System.out.println(stack.pop());
```

А для обращения с обычной очередью служат методы add() и remove(), как демонстрируется в следующем примере кода:

```
Queue<String> queue = new ArrayDeque<>();
queue.add("Peter");
queue.add("Paul");
queue.add("Mary");
while (!queue.isEmpty())
  System.out.println(queue.remove());
```

Потокобезопасные очереди обычно употребляются в параллельно выполняемых программах. Подробнее об этом речь пойдет в главе 10.

Очередь с приоритетами допускает извлечение ее элементов в отсортированном порядке после их ввода в произвольном порядке. Это означает, что всякий раз, когда вызывается метод remove(), получается наименьший элемент, находящийся в настоящий момент в очереди с приоритетами.

Очередь с приоритетами чаще всего применяется для планирования заданий, как показано в приведенном ниже примере кода. У каждого задания имеется свой приоритет. Задания вводятся в такую очередь в произвольном порядке. Всякий раз, когда новое задание запускается на выполнение, из очереди удаляется задание с наивысшим приоритетом. (По традиции наивысший приоритет обозначается числом 1, и поэтому в результате вызова метода remove() из очереди извлекается наименьший элемент.)

```
public class Job implements Comparable<Job> { ... }
...
PriorityQueue<Job> jobs = new PriorityQueue<>();
jobs.add(new Job(4, "Collect garbage"));
jobs.add(new Job(9, "Match braces"));
jobs.add(new Job(1, "Fix memory leak"));
...
while (jobs.size() > 0) {
  Job job = jobs.remove(); // Наиболее срочные задания извлекаются
                           // из очереди первыми
  execute(job);
}
```

Аналогично древовидному множеству типа TreeSet, очередь с приоритетами может содержать элементы класса, реализующего интерфейс Comparable. С другой стороны, конструктору такого класса может быть передан интерфейс Comparator. Но в отличие от древовидного множества типа TreeSet, перебор элементов очереди с

приоритетами совсем не обязательно приводит к их получению в отсортированном порядке. В очереди с приоритетами применяются алгоритмы для ввода и удаления из нее элементов, в результате чего наименьший элемент тяготеет к корню очереди и экономится время на сортировку всех элементов.

7.5.5. Слабые хеш-отображения

Класс `WeakHashMap` предназначен для решения следующей интересной задачи: что произойдет со значением, ключ которого не употребляется нигде больше в программе? Если последняя ссылка на ключ исчезнет, то обратиться к объекту-значению не удастся, и поэтому такой объект должен быть удален сборщиком "мусора".

Но сделать это не так-то просто. Ведь сборщик "мусора" отслеживает действующие объекты. А поскольку объект отображения действует, то действуют и все записи в нем. Они не утилизируются, как, впрочем, и значения, на которые они ссылаются.

Именно эту задачу и решает класс `WeakHashMap`. Его структура данных взаимодействует со сборщиком "мусора" для удаления пар "ключ–значение", когда единственная ссылка на ключ присутствует в записи из хеш-таблицы.

Формально для хранения ключей в классе `WeakHashMap` применяются так называемые *слабые ссылки*. В объекте типа `WeakReference` хранится ссылка на другой объект (в данном случае — ключ к хеш-таблице). Объекты данного типа интерпретируются сборщиком "мусора" как особый случай. Если объект доступен *только* по слабой ссылке, сборщик "мусора" утилизирует его, ставя слабую ссылку в очередь, связанную с объектом типа `WeakReference`. Всякий раз, когда для этого объекта вызывается метод, в слабом хеш-отображении типа `WeakHashMap` проверяется наличие новых поступлений в его очереди слабых ссылок и затем из него удаляются связанные с ними записи.

7.6. Представления

Представление коллекции является легковесным объектом, реализующим интерфейс коллекции, но не хранящим его элементы. Например, методы `keySet()` и `values()` из отображения дают его представления.

Другим примером служит метод `Arrays.asList()`. Так, если массив `a` относится к типу `E[]`, то в результате вызова `Arrays.asList(a)` возвращается обобщенный список типа `List<T>`, поддерживаемый элементами данного массива.

Как правило, в представлении не поддерживаются все операции его интерфейса. Нет, например, никакого смысла вызывать метод `add()` для набора ключей из отображения или списка, возвращаемого методом `Arrays.asList()`. В последующих разделах рассматриваются некоторые разновидности представлений, предоставляемых каркасом коллекций в Java.

7.6.1. Диапазоны

Из списка можно составить представление его подсписка, как показано в следующем примере кода:

```
List<String> sentence = ...;
List<String> nextFive = sentence.subList(5, 10);
```

В этом представлении элементы доступны по индексу в пределах от 5 до 9. Любые изменения в подсписке, в том числе установка, ввод или удаление элементов, оказывают влияние на исходный список.

Для сортируемых множеств и отображений диапазон указывается по нижнему и верхнему пределам, как показано ниже. Как и в методе `subList()`, первый предел указывается включительно, а второй — исключительно.

```
TreeSet<String> words = ...;
SortedSet<String> asOnly = words.subSet("a", "b");
```

Методы `headSet()` и `tailSet()` возвращают поддиапазон без нижнего и верхнего пределов, как, например, в следующей строке кода:

```
NavigableSet<String> nAndBeyond = words.tailSet("n");
```

В методах из интерфейса `NavigableSet` можно выбрать задание каждого предела включительно или исключительно (см. табл. 7.5). А для сортируемого отображения имеются аналогичные методы `subMap()`, `headMap()` и `tailMap()`.

7.6.2. Пустые и одноэлементные представления

В классе `Collections` имеются статические методы для получения неизменяемого пустого списка, множества, сортируемого множества, навигационного множества, итератора, списочного итератора или перечисления (интерфейса, похожего на итератор и унаследованного из версии Java 1.0).

Аналогично имеются статические методы для получения множества или списка с единственным элементом, а также отображения с единственной парой "ключ–значение" (см. табл. 7.3).

Так, если в методе требуется отображение атрибутов и такие атрибуты отсутствуют или имеется только один из них, то можно сделать следующий вызов:

```
doWork(Collections.emptyMap());
```

или такой вызов:

```
doWork(Collections.singletonMap("id", id));
```

вместо того, чтобы создавать тяжеловесный объект типа `HashMap` или `TreeMap`.

7.6.3. Неизменяемые представления

Иногда содержимое коллекции требуется сделать общедоступным, но не изменять его. Разумеется, значения из исходной коллекции можно скопировать в новую коллекцию, но это потенциально затратная операция. Вместо этого лучше выбрать неизменяемое представление. В качестве примера рассмотрим типичную ситуацию, где в объекте типа `Person` хранится список друзей конкретного лица. Если бы метод `getFriends()` возвратил ссылку на этот список, то его можно было бы изменить в

вызывающем коде. Но для этой цели надежнее предоставить неизменяемое представление следующим образом:

```
public class Person {
  private ArrayList<Person> friends;

  public List<Person> getFriends() {
    return Collections.unmodifiableList(friends);
  }
  ...
}
```

Все модифицирующие методы генерируют исключение, если они вызываются для неизменяемого представления. Как следует из табл. 7.3, неизменяемые представления можно получить в виде коллекций, списков, множеств, сортируемых множеств, навигационных множеств, отображений, сортируемых отображений и навигационных отображений.



НА ЗАМЕТКУ. Как было показано в главе 6, элементы неверного типа можно незаконным путем переместить в обобщенную коллекцию (это явление называется *загрязнением "кучи"*), а ошибка времени выполнения возникнет при попытке извлечь, а не ввести элемент неподходящего типа в такую коллекцию. Чтобы устранить подобную ошибку, следует воспользоваться *проверяемым представлением*. Например, в том месте, где строится списочный массив типа **`ArrayList<String>`**, следует построить проверяемое представление следующим образом:

```
List<String> strings =
     Collections.checkedList(new ArrayList<>(), String.class);
```

Такое представление контролирует все операции ввода элементов в список и генерирует исключение, если в него вводится объект неверного типа.



НА ЗАМЕТКУ. Средствами класса **`Collections`** можно также получать *синхронизированные* представления, обеспечивающие надежный параллельный доступ к структурам данных. На практике такие представления не настолько полезны, как структуры данных из пакета **`java.util.concurrent`**, специально предназначенные для параллельного доступа. Поэтому лучше пользоваться классами из данного пакета, держась подальше от синхронизированных представлений.

Упражнения

1. Реализуйте алгоритм под названием "Решето Эратосфена" для выявления всех простых чисел меньше или равных ***n***. Сначала введите все числа от 2 до ***n*** в множество. Затем организуйте повторяющийся поиск наименьшего элемента ***s*** в этом множестве, постепенно удаляя из него элементы s^2, $s \cdot (s + 1)$, $s \cdot (s + 2)$ и т.д. Этот поиск завершается при условии, когда $s^2 > n$. Реализуйте данный алгоритм как для множества типа `HashSet<Integer>`, так и для множества типа `BitSet`.
2. Сделайте все буквы прописными в символьных строках, содержащихся в массиве. С этой целью воспользуйтесь сначала итератором, затем перебором индексных значений в цикле и, наконец, методом `replaceAll()`.

3. Как вычислить объединение, пересечение и разность двух множеств, используя только методы из интерфейса `Set`, но не организуя циклы?
4. Воспроизведите ситуацию, когда возникает исключение типа `ConcurrentModificationException`. Что можно предпринять, чтобы избежать этого?
5. Реализуйте метод `public static void swap(List<?> list, int i, int j)`, выполняющий перестановку элементов обычным образом, когда класс, определяющий тип параметра `list`, реализует интерфейс `RandomAccess`, а иначе сводящий к минимуму обход элементов на позициях, обозначаемых индексами `i` и `j`.
6. В этой главе рекомендовалось пользоваться интерфейсами вместо конкретных классов структур данных, например, интерфейсом `Map` вместо класса `TreeMap`. К сожалению, подобная рекомендация не идет дальше этого. Почему нельзя, например, воспользоваться интерфейсом `Map<String, Set<Integer>>`, чтобы представить содержание документа? (*Подсказка:* как инициализировать его?) Каким типом структуры данных можно вместо этого воспользоваться?
7. Напишите программу для чтения всех слов из файла и вывода частоты, с которой каждое слово встречается в нем. Воспользуйтесь для этой цели классом `TreeMap<String, Integer>`.
8. Напишите программу для чтения всех слов из файла и вывода строк, в которых каждое слово встречается в нем. Воспользуйтесь для этой цели преобразованием из символьных срок в множества.
9. Счетчик в отображении счетчиков можно обновить следующим образом:

   ```
   counts.merge(word, 1, Integer::sum);
   ```

   Сделайте то же самое без метода `merge()`, воспользовавшись, во-первых, методом `contains()`; во-вторых, методом `get()` и проверкой пустых значений (`null`); в-третьих, методом `getOrDefault()` и, в-четвертых, методом `putIfAbsent()`.
10. Реализуйте алгоритм Дейкстры для поиска кратчайших путей между городами, связанными сетью автомобильных дорог. (Описание этого алгоритма можно найти в популярной литературе по алгоритмам или в соответствующей статье Википедии.) Воспользуйтесь вспомогательным классом `Neighbor` для хранения названия соседнего города и расстояния до него. Представьте полученный граф в виде преобразования названий городов в множества соседних городов. Воспользуйтесь в данном алгоритме классом `PriorityQueue<Neighbor>`.
11. Напишите программу для чтения предложения в списочный массив. Затем перетасуйте в массиве все слова, кроме первого и последнего, используя метод `Collections.shuffle()`, но не копируя слова в другую коллекцию.
12. Используя метод `Collections.shuffle()`, напишите программу для чтения предложения, перетасовки его слов и вывода результата. Устраните (до и после перетасовки) написание начального слова с заглавной буквы и наличие точки в конце предложения. *Подсказка:* не перетасовывайте при этом слова.

13. Всякий раз, когда в отображение типа `LinkedHashMap` вводится новый элемент, вызывается метод `removeEldestEntry()`. Реализуйте подкласс `Cache`, производный от класса `LinkedHashMap` и ограничивающий размер отображения заданной величиной, предоставляемой в конструкторе этого класса.
14. Напишите метод для получения неизменяемого представления списка чисел от 0 до n, не сохраняя эти числа.
15. Обобщите предыдущее упражнение произвольным функциональным интерфейсом `IntFunction`. Имейте в виду, что в конечном итоге может получиться бесконечная коллекция, поэтому некоторые методы (например, `size()` и `toArray()`) должны генерировать исключение типа `UnsupportedOperationException`.
16. Усовершенствуйте реализацию из предыдущего упражнения, организовав кеширование последних 100 значений, вычисленных функцией.
17. Покажите, каким образом проверяемое представление может сообщить о конкретной ошибке, ставшей причиной загрязнения "кучи".
18. В классе `Collections` имеются статические константы `EMPTY_LIST`, `EMPTY_MAP` и `EMPTY_SET`. Почему они не так полезны, как методы `emptyList()`, `emptyMap()` и `emptySet()`?